Heaps, Stacks and Queues

Topic 4 of 10

11 questions | skeleton and writing space only

Each question: the prompt, the function or class signature, and ruled lines to write your own solution.

1. Active Communication Top Users

MEDIUM | asked 2026-09-22 | hashmap, set, sorting, top-k, streaming

Process a continuous stream of user pairs such as A-B and B-C; each pair creates an active communication edge, and the system should output users ordered by current communication count. An additional API variant exposes register and mostActiveUser(), then asks for constant-time most-active lookup and a changing-K ranking extension.

```
class Active:

    def __init__(self, count_self=False):

    def bump(self, user):

    def add(self, a, b):

    def most_active(self):

    def top_k(self, k):
```

2. Streaming Median (LC 295) with Follow-ups

MEDIUM | asked 2026-08-03 | heap, streaming

A near-canonical LC 295 streaming-median implementation using two heaps, followed by two design extensions: exploit a very small value domain, then support an arbitrary k-th-smallest query instead of only the median.

```
class MedianFinder:

    def __init__(self):

    def add(self, x):

    def median(self):
```

3. Batch and Streaming Log Deduplication

MEDIUM | asked 2026-07-23 | dedup, streaming, heap, sorting

Process (timestamp, message) logs by deduplicating on message and sorting retained entries by timestamp, first keeping the first-seen entry and then the latest timestamp. The streaming extension adds add(timestamp, message) and get_next(), requiring a map plus min-heap, lazy deletion, and a discussion of watermarks and bounded state.

```
def dedupe(logs, policy='first'):
```

4. Top-K Frequent in Large Logs

MEDIUM | asked 2026-07-19 | heap, hashmap, scaling

Examples: Input: chatLog = ["A says hi", "B says hi", "A says bye", "C says hi", "A says yo"], k=2 Counts: A=3, B=1, C=1 Output: [A, B] // or [A, C] -- ties allowed

```
def top_k_speakers(chat_log, k):
```

5. Median of Two Sorted Arrays (LC 4) with a Follow-up Variant

HARD | asked 2026-10-09 | binary-search, heap, divide-and-conquer

L5 R1 coding: LC 4 (Median of Two Sorted Arrays).

```
def median_two(a, b):
```

6. Visible People in a Line (Last Person, Then Everyone)

MEDIUM | asked 2026-10-07 | array, monotonic-stack, stack

Given heights of people in a line, A can see B if nobody between them is taller than both. Part 1: count how many people the last person can see. Part 2: return that count for every person. Written in a plain editor with no compilation. Input: an array of heights representing a line of people. Visibility rule: person A can see person B if no one standing between them is taller than both A and B. Part 1: return how many people the last person in line can see.

```
def last_sees(a):
```

7. Decode String

MEDIUM | asked 2026-07-29 | string, stack, parsing

Examples: "3[a]2[bc]" -> "aaabcbc". "2[abc]3[cd]ef" -> "abcabccdcdcdef". "3[a2[c]]" -> "accaccacc".

```
def decode(s):
```

8. Nested Function Expression Evaluator

MEDIUM | asked 2026-06-10 | parsing, recursion, stack

Examples: add(1, sub(1, 0)) -> 2 add(2, mul(3, pow(4, 5))) -> 3074

```
def evaluate(expr, env=None):
```

9. Meeting Rooms / Interval Scheduling Variants

MEDIUM | asked 2026-09-29 | interval, heap, greedy

LC 253 (Meeting Rooms II) appears in many forms across phone screen and onsite coding rounds -- minimum number of rooms, room assignment, cyclic time, daily server jobs, gate entry/exit. The base task in every variant: given a list of intervals (start, end) -- or pickup/return pairs, meeting time pairs, server job runs -- answer one or more of: Min rooms / cars / servers needed to honor all intervals concurrently (the canonical min-meeting-rooms problem).

```
def min_rooms(intervals):
```

10. Union of Sorted Interval Lists

MEDIUM | asked 2026-08-16 | interval, two-pointer, heap, streaming

Reverse of LC 986: given two individually sorted, internally non-overlapping interval lists, return their union.

```
def union_two(a, b, touching=True):
```

11. Implement Python's itertools.tee

MEDIUM | asked 2026-08-20 | iterator, queue, data-structure, object-design

Implement the behavior of Python's tee: accept one iterator and an integer n, then return n independent iterators over the same source sequence. Define the required function and classes yourself and improve a naive design toward an efficient shared-state implementation.

```
def tee_queues(iterable, n=2):
```